

Playing with the Rubik's Cube and its God's number in Rocq

Laurent Théry

INRIA, Stamp Team
Laurent.Theiry@inria.fr

Abstract. One needs no more than 20 turns to solve any Rubik's cube position. This is called God's number and was computed in 2010. This note presents 3 smaller results about the Rubik's cube that are proved in Rocq. First, we prove that 20 is a lower bound. Second, we change metric (counting half turns as 2 moves) and prove that in this case 26 is a lower bound. Finally, the published computation of God's number (35 CPU years) splits the cube into the 2 217 093 120 cosets of a subgroup, one search to a coset. We formalise the correctness of the coset search, and certify the computation on one such coset.

Keywords. Rubik's cube, God's number, formal proof, Rocq, group theory.

1 The problem

A Rubik's cube is built from 26 small cubes: 8 corners with 3 stickers each, 12 edges with 2 stickers each and 6 centres. The centres do not move. A turn moves 4 corners and 4 edges, and rotates a centre.

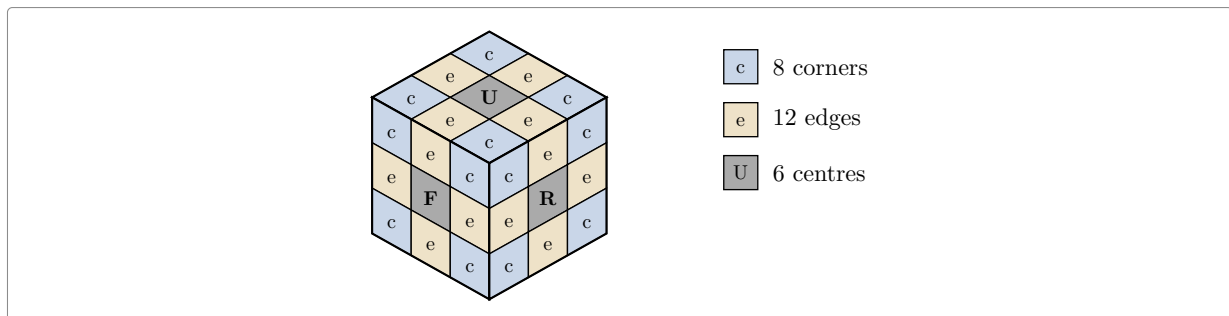


Figure 1: Anatomy of the cube

Not every arrangement of the pieces can be reached by turning faces. The number of possible arrangements is

$$8! \cdot 3^7 \cdot 12! \cdot 2^{11} / 2 = 43\,252\,003\,274\,489\,856\,000 \approx 4.3 \cdot 10^{19}.$$

The 8 corners can be in any order ($8!$). Their orientations are free, except for the last one (3^7). The same holds for the 12 edges ($12!$ and 2^{11}). The result is divided by 2, because corners and edges are not independent.

Turning one face is a *move*. A half turn counts as one move, like a quarter turn. The number of moves needed by the worst position is called *God's number*. Counting a half turn as one move is a choice. A half turn can also count as 2 moves. This gives a second number for the same cube. We prove a lower bound for each.

We take one position: the **superflip**. Every small cube is at its right position, but all the edges have the wrong orientation.

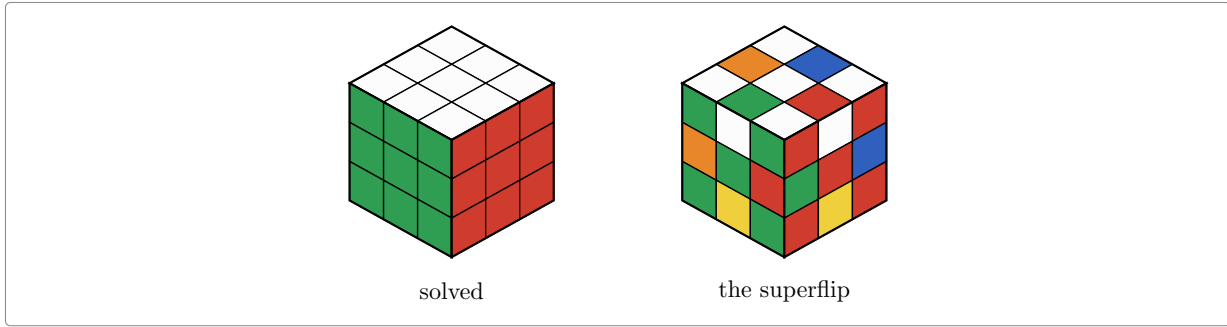


Figure 2: A solved cube, and the superflip.

The superflip is left unchanged by all 48 symmetries of the cube. We use it to prove the lower bound.

2 The cube as permutations

There are several ways to represent the Rubik's cube. We use stickers. There are 6 faces of 9 stickers. We remove the 6 centre stickers, which never move. The other 48 stickers are numbered from 0 to 47, as in Figure 3 and Figure 4. A move is then a rearrangement of the 48 stickers. Each face has 8 of them, taken left to right and top to bottom, with the centre skipped. Up gets 0–7, left 8–15, front 16–23, right 24–31, back 32–39 and down 40–47.

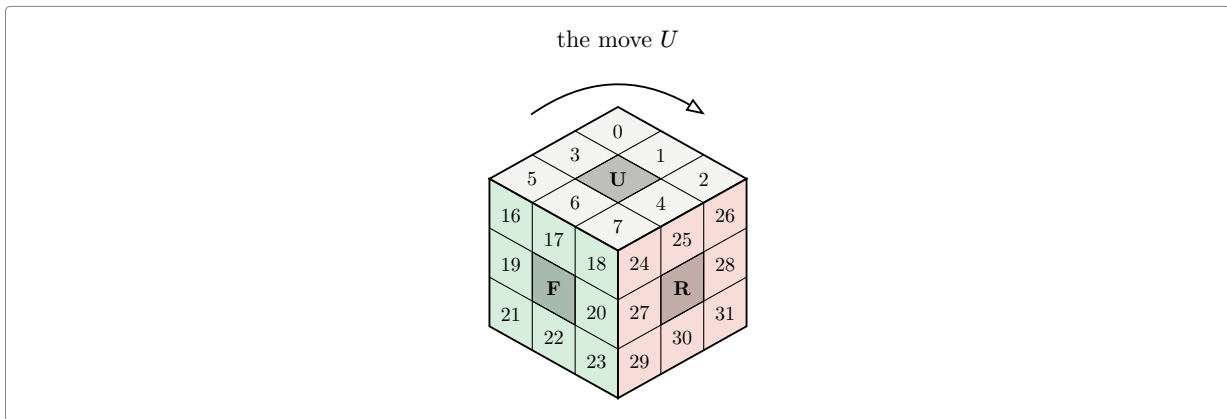


Figure 3: The top, front and right faces, and the turn of the top one.

			0	1	2			
			3	U	4			
			5	6	7			
8	9	10	16	17	18	24	25	26
11	L	12	19	F	20	27	R	28
13	14	15	21	22	23	29	30	31
			40	41	42			
			43	D	44			
			45	46	47			

Figure 4: The cube unfolded, with all 48 places numbered.

A move sends each sticker to a new place. For example, if we turn the top face clockwise, the sticker in corner 0 goes to corner 2, the one in 2 to 7, the one in 7 to 5 and the one in 5 back to 0. This

is a cycle of 4 steps, written (0 2 7 5). The 4 edge stickers of that face do the same: (1 4 6 3). The turn also moves the top row of each side face to the next side face: front to left, left to back, back to right, right to front. This gives 3 more cycles, (8 32 24 16) and 2 similar ones. Figure 5 shows the whole move. Each square shows the sticker that is there after the turn. For example, the top row of the left face now holds 16, 17 and 18, the stickers that came from the front.

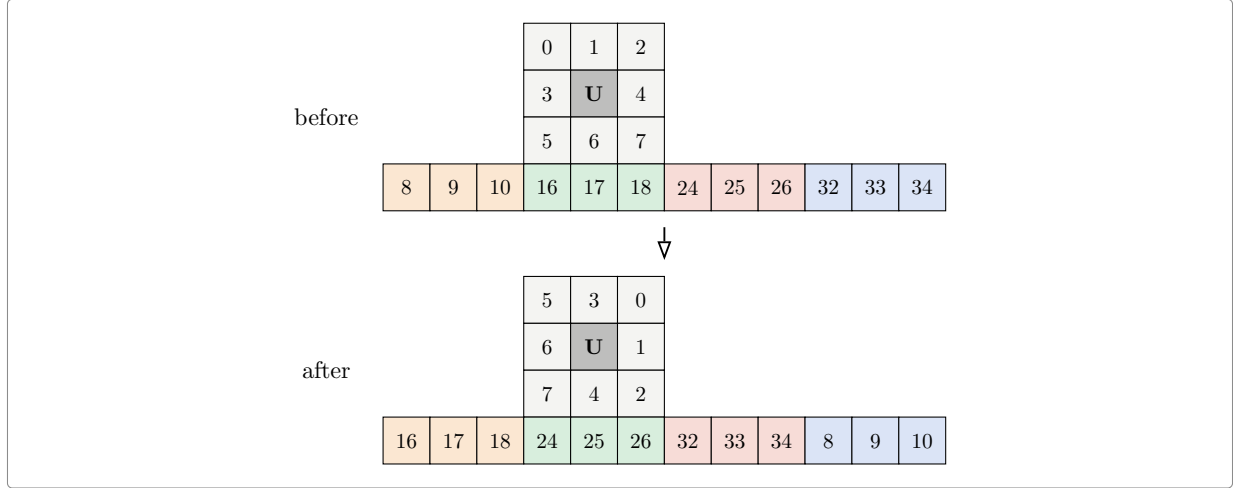


Figure 5: Everything a clockwise turn of the top face moves.

There are 6 clockwise quarter turns: up, right, front, down, left and back. Each of them can also be done twice or backwards. This gives the *18 moves*:

U	R	F	D	L	B
U2	R2	F2	D2	L2	B2
U'	R'	F'	D'	L'	B'

A move done *twice* is written U2, and a move done *backwards* is written U'. A position is a product of moves, written as a *word*. For example, R U R' U' is a word of length 4. The set of all positions is a group *G*, the *cube group*. A position solved in *d* moves is a word of *d* moves. The positions solved in at most *d* moves form the *ball of radius d* around the solved cube. God's number is the diameter of the Cayley graph.

2.1 The cube in Rocq

We use the Rocq prover [1] and its Mathcomp library [2]. The file `Rubik333.v` is a direct translation of the paragraphs above:

Definition facelet := 'I_48.

Definition Umove : {perm facelet} := cyc [:: 0@; 2@; 7@; 5@] * cyc [:: 1@; 4@; 6@; 3@] * cyc [:: 8@; 32@; 24@; 16@] * cyc [:: 9@; 33@; 25@; 17@] * cyc [:: 10@; 34@; 26@; 18@]. (* ... and five more, one per face ... *)

Definition faces : seq {perm facelet} :=
[:: Umove; Rmove; Fmove; Dmove; Lmove; Bmove].

Definition moves : seq {perm facelet} :=
flatten [seq [:: g; g ^+ 2; g ^-1] | g <- faces].

Definition G : {group {perm facelet}} := <<Sset>>.

Line by line:

- 'I_48 is the type of numbers below 48. It is a finite type.
- {perm facelet} is the type of *permutations* of those facelets.

- `cyc [:: 0@; 2@; 7@; 5@]` is the *cycle* that sends 0 to 2, 2 to 7, 7 to 5 and 5 back to 0, leaving the other 44 places where they are. The @ is local notation turning a plain number into a facelet.
- The symbol `*` composes 2 permutations. So `Umove` is the cycles of Figure 5 put together. `g ^+ 2` is the turn done twice, and `g ^-1` the turn undone. The order of composition is the opposite of the usual one. Mathcomp applies permutations on the right, so `(g * m) f` is `m (g f)`.
- `seq` is a list. `faces` is the list of the 6 clockwise quarter turns. `moves` takes 3 moves for each face, which gives the 18 moves.
- `<<Sset>>` is the group generated by the moves: all the positions that we can reach by composing moves. This is the cube group.

The superflip is defined in `Diameter.v`. It is made of 12 swaps, one for each edge, that exchange the 2 stickers of the edge:

```
Definition Spcyc : seq (seq facelet) :=
  [:: [:: 1@; 33@]; [:: 3@; 9@]; [:: 4@; 25@];
    (* ... nine more, one per edge ... *) ].
```

```
Definition superflip : {perm facelet} := \prod_(l <- Spcyc) cyc l.
```

This makes it a permutation, but not yet a legal position. For this, we have to prove that it belongs to G . The proof is one equality, with this word of 20 moves:

U R2 F B R B2 R U2 L B2 R U' D' R2 F R' L B2 U2 F2

3 Searching for the lower bound

We want to show that the superflip cannot be solved in 19 moves or less. The simple approach is to try every word of at most 19 moves from the superflip. If the solved cube never appears, 20 is a lower bound. But a tree of depth 19 with 18 branches at each node has 18^{19} words. This is far too many. We need to refine the search.

3.1 The pruning estimate

A first refinement is a test that cuts early the branches that cannot succeed. For this, we give each position a rough lower bound h on the number of moves still needed. If h is 20 and only 18 moves remain, the branch cannot reach the solved cube in time. It is cut, with everything below it, as Figure 6 shows.

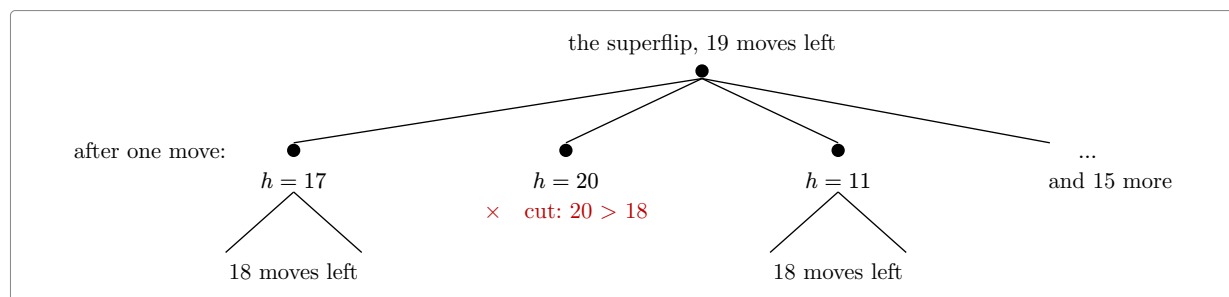


Figure 6: The search, and its scissors.

Such a lower bound is called an **admissible** estimate. A depth-first search that increases its depth step by step and cuts with such an estimate is Korf's IDA* [3].

3.2 Getting a cheap estimate

The idea behind our estimate is to forget most of the cube. We keep only part of the information, for example how the corners are twisted and where the 4 middle-layer edges are. We call what is left a *summary*. Many positions share the same summary, and they get the same estimate. Moves act on summaries as well as on cubes. There are few summaries, so we can precompute the exact distance

of each one to the solved summary, and store it in a table. This gives h : take a position, compute its summary and read its distance in the table. The idea comes from Culberson and Schaeffer, who call such a table a *pattern database* [4]. Korf solved the cube optimally with 3 of them [5].

We use Kociemba's summary, from his two-phase solver [6]. We call it the *phase 1 summary*. Figure 7 shows the 3 things it records. A corner has one sticker of the up or down face. This sticker is in one of 3 places, numbered 0, 1 or 2. An edge is either the right way round or turned over, numbered 0 or 1. The 4 edges of the middle layer are in 4 of the 12 edge slots. The figure shades these slots on the 2 visible faces, where 3 of the 4 can be seen. Nothing else is recorded.

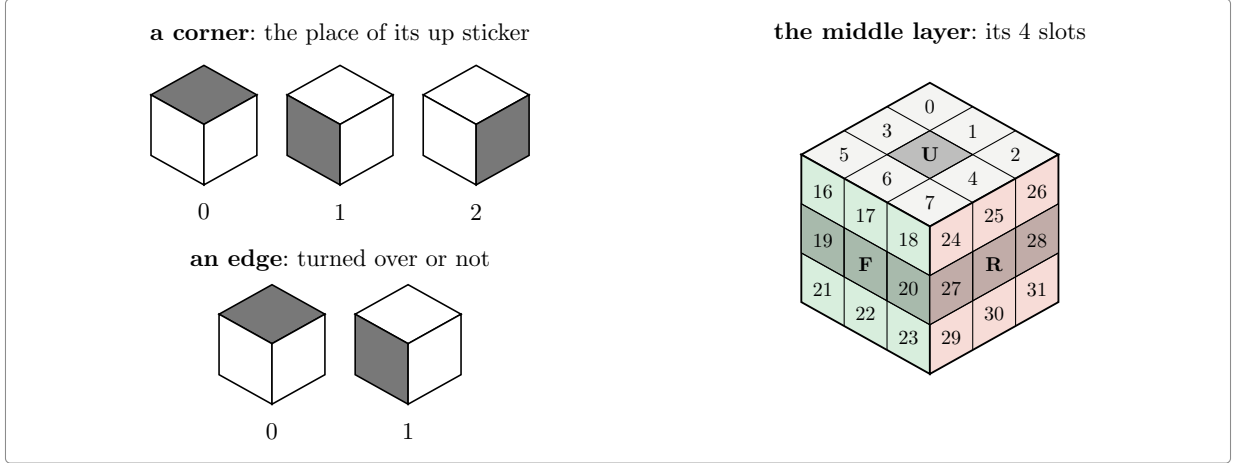


Figure 7: The 3 things a summary records.

The summary is the product of the 3 values:

summary	values	
how the 8 corners are twisted	2 187	$= 3^7$
how the 12 edges are flipped	2 048	$= 2^{11}$
where the 4 middle-layer edges sit	495	4 places among 12
the summary	2 217 093 120	

Every summary stands for exactly 19 508 428 800 positions. The table gives, for each summary, its distance from the solved summary. A distance is never more than 12, so 4 bits hold one entry, and a 63-bit machine word holds 15 entries. The whole table is *1.18 GB*. The cut is very effective. A search at depth 14 visits only 445 398 nodes. Without the cut, the same tree has $1.03 \cdot 10^{15}$ nodes.

3.3 Searching in Rocq

The search is generic. It is defined in the file `Search.v`. It takes a group, a set of moves, an estimate h and 2 hypotheses on h :

Hypothesis `h1` : $h \ 1 = 0$.

Hypothesis `hstep` : $\text{forall } g \ m, m \in S \rightarrow h \ g \leq (h \ (g * m)).+1$.

Fixpoint search ($d : \text{nat}$) ($g : gT$) : $\text{bool} := (h \ g \leq d) \ \&\& \ ((g == 1) \ || \ (\text{if } d \text{ is } d'.+1 \text{ then has } (\text{fun } m \Rightarrow \text{search } d' \ (g * m)) \ \text{Sseq else false}))$.

Corollary `searchN d g` : $\text{search } d \ g = \text{false} \rightarrow g \notin \text{ball } S \ d$.

Line by line:

- `gT` is the group. It is then instantiated with the cube group.
- `1` is the unit of that group, the solved position for the cube. So with $h \ 1 = 0$ we ask the estimate to be 0 on the solved cube, and $g == 1$ tests whether the search has reached the solved position.

- `Sseq` is the list of the 18 moves, and `S` is the same thing seen as a set.
- `g * m` is the position `g` followed by the move `m`.
- `h g <= d` is the cut. `(h (g * m)).+1` is the estimate after `m` plus one, so by `hstep` one move changes the estimate by at most one.
- `has (fun m => search d' (g * m)) Sseq` tries every move with one fewer move left, and answers as soon as one of them succeeds.

The last line is what we need. If the search returns false, the position is not in the ball of radius d . The next subsections describe the improvements that give our final search.

3.4 Searching with a summary

The estimate is built in a second generic file, `Coord.v`. It takes the summary of a position, the action of a move on a summary and the table of distances. It asks one condition on the summary and 2 on the table.

```
Variable coord : {perm facelet} -> X.
Variable act    : X -> {perm facelet} -> X.
Hypothesis coordM : forall g m, coord (g * m) = act (coord g) m.
```

```
Variable D : X -> nat.
Hypothesis D0 : D (coord 1) = 0.
Hypothesis Dstep : forall x m, m \in Sset -> D x <= (D (act x m)).+1.
```

`X` is the type of summaries, and `coord` gives the summary of a position. `act` applies a move directly to a summary, without going back to the position. By `coordM`, the two agree: applying a move and then taking the summary gives the same result as `act`.

The search of `Search.v` is refined to carry the summary with the position. The summary is updated with `act` at each move. The position is still needed, to test whether the cube is solved. Here is the refined search, with simpler names and some details left out. The real one is `searchz3` in `Farpl.v`.

```
Fixpoint search (d : nat) (g : gT) (x : summary) (p : move) : bool :=
  (D x <= d) &&
  ((g == 1) ||
   (if d is d'.+1
    then has (fun m => search d' (g * m) (act x m) m) (allowed p)
    else false)).
```

Here are some explanations:

- `D x <= d` is the cut. It reads the table at the summary `x`.
- `g == 1` tests whether the cube is solved.
- `g * m` moves the position and `act x m` moves the summary. The summary never has to be recomputed from the position.
- `p` is the last move, and `allowed p` is the list of moves allowed after it. This is explained in section 3.6.

3.5 Verifying the table

For the phase 1 summary, `D` is a lookup in the phase 1 table, so `D0` and `Dstep` become 2 statements about that table:

- the entry of the solved summary is 0;
- every entry is at most one more than the entry reached from it by any of the 18 moves.

The table is produced by an OCaml program, and it gives the exact distance. But the table is not trusted. The 2 statements above are checked in Rocq by computation. A table full of zeros would also pass these checks, but then the search would cut nothing. The entries of the table do not depend on each other. So the second check, the most expensive one, is cut into slices, one file for each slice. The files are checked in parallel, since Rocq compiles files separately. This takes 10 minutes. All the

timings in this note are measured on the same machine, the *reference machine*: a dual-socket Intel Xeon E5-2667 at 2.9 GHz, with 12 cores, 24 threads and 62 GB of memory.

3.6 Removing redundant moves

Many words lead to the same position. We use 2 ideas to leave some of them out. The first idea is to avoid repetition. For example, after a U we do not need to try U , U^2 or U' . Shorter words, at a smaller depth, already reach these positions. This leaves 15 moves instead of 18. Opposite faces give a weaker form of the same argument. $U D$ and $D U$ give the same position, so we keep only one of the 2 orders. We privilege the top, right or front face first. We call this the *order convention*. From the second move on, it leaves 15 moves after a turn of the top, right or front face, and 12 after a turn of the bottom, left or back face. The second idea is symmetry. The superflip is unchanged by all 48 symmetries of the cube. So for the first move we only need the turns of one face. We choose the top face. By symmetry again, we only need U and U^2 , since U' is the mirror image of U . The search is then parallelised at depth 2: 2 first moves times 15 second moves is 30 *prefixes*, and each prefix is searched in a file of its own.

3.7 Composing summaries

We can turn the whole cube around the axis through 2 opposite corners. We get the same position, seen from another side. Its summary gives another entry of the same table. So each position has 3 summaries, and the maximum of the 3 values is still an admissible estimate. This costs 3 reads of the table instead of one. But it is worth it, because the tree is much smaller.

3.8 Folding the table

The summary is built around the up-down axis. Of the 48 symmetries, 16 keep this axis. They turn one summary into another. They sort the 1 013 760 values of flip and slice into 64 430 families. One entry for each family is enough, so the table can be made smaller. In the code, the change is one definition. The lookup was

```
Definition Dpli (tw x : int) : int := plget (plidx tw x).
```

and it became

```
Definition Dplri (tw r : int) : int :=
  plget (plfoldi (frep r) (twsym tw (fsym r))).
```

`frep r` is the representative of the family. `fsym r` is the symmetry that sends the value to it. `twsym tw (fsym r)` is the twist sent by the same symmetry. Each lookup costs 3 more reads, in a table 15.73 times smaller.

Tables reduced by symmetry are common in cube solvers. With the fold, the search is 1.61 times slower at depth 16. But a search worker goes down from 4.15 GB to 3.6 GB, so 18 of them can run at the same time. And checking the table goes down from about 5.4 processor hours to 1.35.

3.9 Conclusion on the first lower bound

The development for this lower bound has 44 files: 9 about the cube (2 500 lines), 15 about the search (4 900 lines) and 20 about the tables (5 200 lines). Building the tables costs the same for any radius of the search. Here are the times, measured from a clean tree on the reference machine:

	wall clock	processor time
emitting the tables and compiling them to native code	17 min 21	52 min 13
the coordinate and summary tables	22 min 46	21 min 15
the search and the estimate, over no data at all	11 min 02	30 min 45
the 4 certificates for the move and distance tables	56 s	2 min 27
the fold: 12 checks and 27 slices	9 min 52	47 min 57
the shape check against the dummy table	18 s	16 s

	wall clock	processor time
in total	1 h 02	2 h 35

The second line runs on one core, so more cores do not help. The first and fifth lines are mostly the OCaml compiler, turning a table into native code. Together they take 100 of the 155 processor-minutes.

The search visits 137 607 893 106 positions. The smallest piece has 3 067 879 204 and the largest 8 527 685 275. Between depths 17 and 19, the tree grows by 12.22 from one level to the next. We measured the run twice, once before the fold and once after. The theorem is the same both times.

radius 19, search depth 17	before	after
pieces	18	30
workers	9	18
memory per worker	4.15 GB	3.6 GB
wall clock	11 h 13	5 h 48
processor time	85 h 11	89 h 27

The 30 pieces balance the run. Each piece is one prefix of 2 moves. They are compiled longest first, since make starts them in the order it is given. The pieces are far from equal. The longest takes 5 h 43 of processor time, and the shortest 1 h 43. So the longest piece sets the wall clock. The wall clock, 5 h 48, is only 5 minutes more than the best that any order of these 30 pieces can give.

4 Counting in quarter turns

We have proved that God's number is at least 20. We now count the moves differently. In quarter turns there are 12 moves: the 6 faces turned one way, and the same 6 turned back. A half turn counts as 2 moves. In this count, the answer is **26** (cube20.org).

4.1 The position

The superflip is only 24 quarter turns from solved, so it is not far enough. Reid posted a better position to the Cube-Lovers list in August 1998 [7]: the superflip composed with four spot, the four-spot pattern with the superflip on top of it. In the following, we call it **superflip4**.

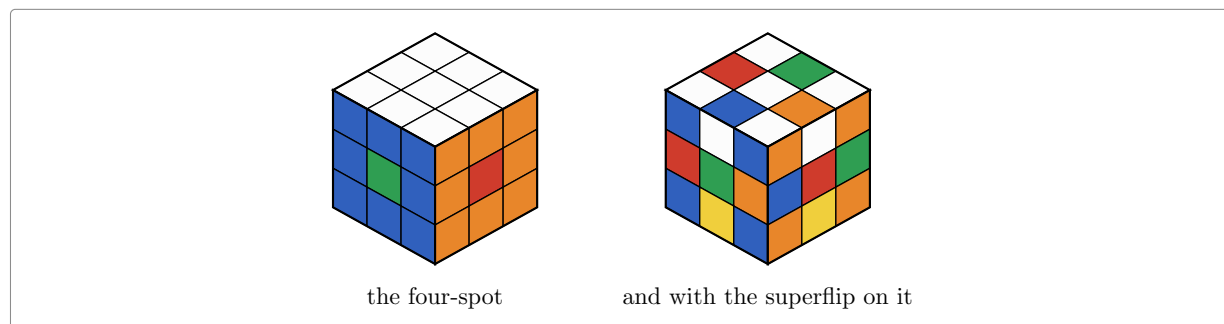


Figure 8: The four-spot, and superflip4.

Superflip4 is 26 quarter turns from solved. Here is the word given for it, with the half turns written out:

U U D D L F F U' D R R B U' D' R L F F R U D' R' L U F' B'

We want to prove that this position cannot be solved in 25 quarter turns. The parity of permutations saves one level of search (24 instead of 25). A cycle of 4 stickers is the product of 3 swaps, so it is an odd permutation. A quarter turn is 5 cycles of 4 stickers, so it is odd too. So a word of even length gives an even permutation, and a word of odd length gives an odd one. This argument is in **HBound.v**.

Because of this word of 26 moves, superflip4 is even. So all its words have an even length. If superflip4 has no word of at most 24 moves, it is exactly 26 moves from solved.

4.2 The 6 prefixes

Every shortest word for superflip4 can be rewritten, at the same length, so that it begins with one of 6 prefixes [7]:

$$\begin{array}{ccc} \text{R U} & \text{R' U D} & \text{R' U F'} \\ \text{R' U R'} & \text{R' U B'} & \text{R' U L'} \end{array}$$

The searches start from those 6. In order to prove this fact, we split the 12 quarter turns into 2 sets:

$$\mathcal{A} = \{ \text{U, U', D, D'} \}, \quad \mathcal{C} = \{ \text{R, R', F, F', L, L', B, B'} \}.$$

A word made only of turns from $\mathcal{A}$ leaves the 4 middle edges untouched, and superflip4 has them flipped, so no such word gives superflip4. A word made only of turns from $\mathcal{C}$ never flips an edge, and superflip4 has every edge flipped, so no such word gives it either. Any word for superflip4 therefore uses turns from both sets, so somewhere in it there is a two-letter subword made of a letter of $\mathcal{A}$ and a letter of $\mathcal{C}$.

Each of the 6 prefixes starts with a turn from $\mathcal{C}$ followed by a turn from $\mathcal{A}$, so we want a word for superflip4 that starts with such a subword. Recall the word we gave for it:

$$\text{U U D D L F F U' D R R B U' D' R L F F R U D' R' L U F' B'}$$

Reading along it, 4 two-letter subwords have the right order and 5 have the wrong order. We take the first one with the right order, F U' , and call it q . A word may have had only subwords with the wrong order. In that case, we would then invert it first: superflip4 is its own inverse, so the inverse is again a word for it, of the same length, and the orders are swapped. Here the inverse is

$$\text{B F U' L' R D U' R' F' F' L' R' D U B' R' R' D' U F' F' L' D' D' U' U'}$$

with, as expected, 5 subwords in the right order and 4 in the wrong order.

Let us go back to our two-letter subword q . It has 6 turns before it. Write x for them, U U D D L F , and w for what follows q , so that $xqw = P$, where P is superflip4. Then $qw = x^{-1}P$, and therefore

$$qw(P^{-1}xP) = x^{-1}PP^{-1}xP = P.$$

$P^{-1}xP$ is not longer than x :

$$P^{-1}xP = (P^{-1}\text{U}P)(P^{-1}\text{U}P)(P^{-1}\text{D}P)(P^{-1}\text{D}P)(P^{-1}\text{L}P)(P^{-1}\text{F}P) = \text{U U D D R B}.$$

The first step is done: qw followed by those 6 turns is a word for superflip4 of the same length, beginning with q .

$$\text{F U' D R R B U' D' R L F F R U D' R' L U F' B' U U D D R B}$$

The second step uses symmetries. A symmetry of the cube that leaves superflip4 unchanged carries a word for it to another word for it, of the same length. Of the 48 symmetries, 16 do that. The superflip is unchanged by all 48, but the four-spot is not: it leaves the top and bottom faces alone and exchanges the colours of the other 4 in pairs, so it singles out the up-down axis, the line through the centres of the top and bottom faces. A symmetry that moves that axis carries the four-spot to the same pattern about another axis, which is another position. The 16 that keep the axis are the ones that leave superflip4 alone. They make the letter of q from $\mathcal{A}$ into U , and the letter from $\mathcal{C}$ into R or R' , so q becomes R U or R' U . In our example q is F U' . The mirror that swaps left and right fixes the front face and reverses the direction of every turn, so F U' becomes F' U . A quarter rotation about the up-down axis then carries the front face to the right one, and F' U becomes R' U .

A word starting with **R U** needs nothing more, which is the prefix of 2 turns. When it starts with **R' U** we look at the third turn. For 6 of the possible turns, we get words that a symmetry or an inversion brings back to the **R U** case, and 5 do not. Those 5 are the prefixes of 3 turns.

4.3 The summary, and its table

The estimate is built as before. We take the summary Reid uses in the quarter-turn count. Against the phase 1 summary of the first bound, it keeps the corner twist, the same 2 187 values, and replaces the edge flips and the slice by where the 4 middle edges sit with their flips, and by which 4 corner places hold the top corners.

summary	values	
where the 4 middle-layer edges sit, each of them the right way round or not	190 080	$= 24 \cdot 22 \cdot 20 \cdot 18$
which 4 corner places hold the 4 top corners	70	4 places among 8
how the 8 corners are twisted	2 187	$= 3^7$
the 3 together	29 099 347 200	

Each of the 4 edges can sit in any free slot, either way round. That gives 24 choices for the first, and 2 fewer for each of the others, since a slot taken is taken whichever way round the edge in it lies. A summary is the coset of a subgroup H of the cube group, which is why the file names of this section start with H . It is not the subgroup of the first bound, and the summaries are 13 times as many, 29 billion against 2.2 billion.

The table holds the distance from solved of each of the 29 billion summaries. The number of summaries at each distance agrees with the column published in 1998, and we check that first. The table is then folded. The 16 symmetries that keep the up-down axis sort the 190 080 edge values into 12 094 families, a factor of 15.72, and one entry is kept per family. That is 883 MB, and 3.86 GB once loaded into the prover.

4.4 Conclusion on the second lower bound

The word of 26 moves and the 6 searches together show that superflip4 is exactly 26 quarter turns from solved. So in quarter turns, God's number is at least 26. The development for this lower bound has 19 files (6 100 lines). The argument for the 6 prefixes is in **HProp2.v**, the search in **HSearch.v**, the checks in **HSweep.v** and the bound in **HAL1.v**. Here are the times, measured from a clean tree on the reference machine:

	wall clock	processor time
building the table, in OCaml	9 min 50	1 h 43
the same table as 59 Rocq files	3 h 13	6 h 50
checking the 3 move tables		1 min 20
checking the distance table, 12 jobs	2 h 51	31 h 42
the 6 searches, 72 pieces, 12 workers	4 h 00	45 h 54
the whole chain in Rocq	10 h 32	87 h 29

The last row is measured end to end, from a directory where nothing is built. It is not the sum of the others. The OCaml table of the first row is built once by hand, and it is not part of that run. The checks and the searches are nearly nine tenths of the cost. Checking the table, instead of trusting it, costs about two thirds of what the searches cost.

5 One coset of the upper half

So far we have dealt with lower bounds. This section is about the upper bound. Every position of the cube is solved in 20 moves or fewer. The idea of the big computation is the following. Take a subgroup H of G . Every position p lies in exactly one coset xH . So the problem of proving that the

diameter of G is at most 20 is reduced to many independent smaller problems: every position of xH is at most 20 moves from solved. In this note, we are not going to tackle the problem of how the representatives of the cosets are generated. We are going to prove the algorithm that checks that, given an arbitrary x , every position of xH is at most 20 moves from solved. As an application, we then run it inside Rocq with the superflip for x . This shows that all the positions of the superflip's coset are at distance at most 20.

5.1 Marking algorithm

To check that every position of a coset is at most 20 moves from solved, we use a *marking* algorithm. Each position of xH gets one bit. This bit is initially set to 0. The marking works iteratively. For each level d , from 0 to 20, we list the words of length d . A word gives a position. If this position is in xH , we set its bit to 1. We call the bits of a coset its *map*. The bits are stored in machine words. An array in Rocq may hold 4 194 303 entries, so a larger map is split into several arrays. At the end, the bits set to 1 are exactly the positions of xH within 20 moves. If no bit is left at 0, every position of the coset is solved in 20 moves or fewer.

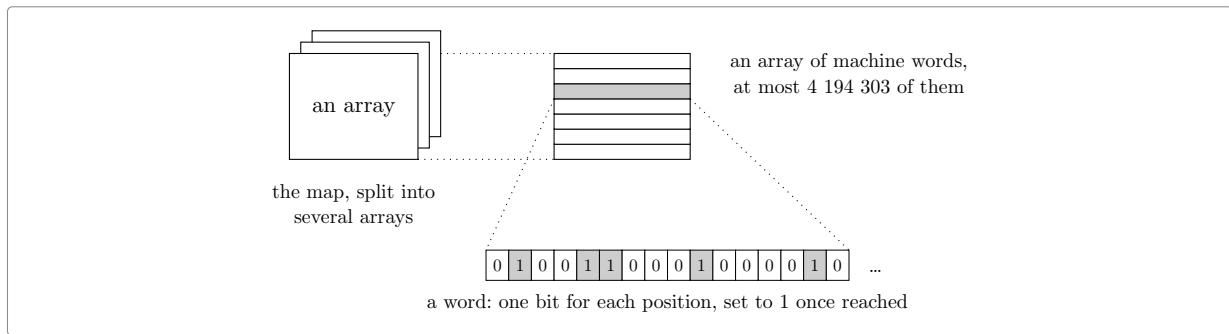


Figure 9: The map of a coset. One bit for each position of the coset, the bits kept in machine words and the words in arrays, as many arrays as Rocq's limit on the length of an array asks for.

5.2 Choosing H

To derive an effective marking algorithm, the choice of the subgroup H is crucial. The one we chose is the one that is associated with the phase 1 summary. It is easy to check that applying 10 of the 18 moves (U, U2, U', D, D2, D', R2, L2, F2 and B2) to a position does not change its summary. In fact, if we take H as the subgroup generated by these 10 moves, the elements of xH are exactly the positions that have the same summary as x . H has other useful properties. H contains exactly 19 508 428 800 positions. So a coset is 19 508 428 800 bits, about 2.4 GB. This fits in the memory of a desktop machine. There are 2 217 093 120 cosets. So up to 2 217 093 120 problems can be run in parallel.

	count
positions of the cube	43 252 003 274 489 856 000
cosets	2 217 093 120
positions in a coset	19 508 428 800

Note that because of symmetries the number of cosets to check can be reduced to 138 639 780. With a further reduction, to 55 882 296 cosets, the whole computation took about 35 CPU years.

Checking membership for xH can be done effectively. A position p is in xH exactly when $x^{-1}p$ is in H , that is, when the summary of $x^{-1}p$ is the solved one. The bit of a position xh of the coset is indexed by h . So we start our enumeration of the words of length d from x^{-1} . We apply the d moves of a word. If the position h we reach is in H , that is, if its summary is the solved one, we set the bit of h .

The phase 1 table gives us the distance to the solved summary. For a position, this is the number of moves needed to bring it into H . We use it to cut the enumeration, as in the search of section 3. Remember that we start from x^{-1} . We build the words of length d one move at a time. Say k moves have been applied to produce $x^{-1}w$, and the table gives t for $x^{-1}w$. If $k + t > d$, no word that continues from there can end in H after d moves: its distance is too high. So we can drop this branch.

Membership is not checked as a distance of 0 in the phase 1 table (the phase 1 table is only proved to be an estimate). Instead it uses the position and its summary. An entry of the phase 1 table also records which moves lower the distance and which keep it (`RowMask.v`), so a node tries 3 or 4 moves instead of 18.

5.3 Structuring the map

An element of H has the solved summary: no corner is twisted, no edge is flipped and the 4 middle edges are in the middle layer. So the element h that indexes a bit can be represented by 3 numbers: how the 8 corners of the top and bottom layers sit, how the 8 edges of those layers sit and how the 4 middle edges sit. That is $40\,320 \times 40\,320 \times 24$ arrangements ($8! = 40\,320$ and $4! = 24$). The corners and the edges are always permuted with the same sign, so this number is divided by 2.

We lay out the bits the way Rokicki's own program lays them out. A page is one arrangement of the corners. Inside a page, a group is a pair of arrangements of the outer edges, the 2 that differ by exchanging 2 edges, and the group's 24 bits are the 24 arrangements of the middle edges, the 12 even ones low and the 12 odd ones high. The parity determines which arrangement of the pair a bit stands for, which is how the impossible half disappears with nothing left to store.

The map uses 48 bits of each machine word, so one word holds the same group on 2 pages, the corner arrangements of even and odd rank, the odd one in the top half. The map is then $20\,160 \times 20\,160$ words: 3.25 GB, organised as 194 arrays of 2 097 152 words, the last one not full. The map and its indexing are defined in `Row.v` and `RowMap.v`.

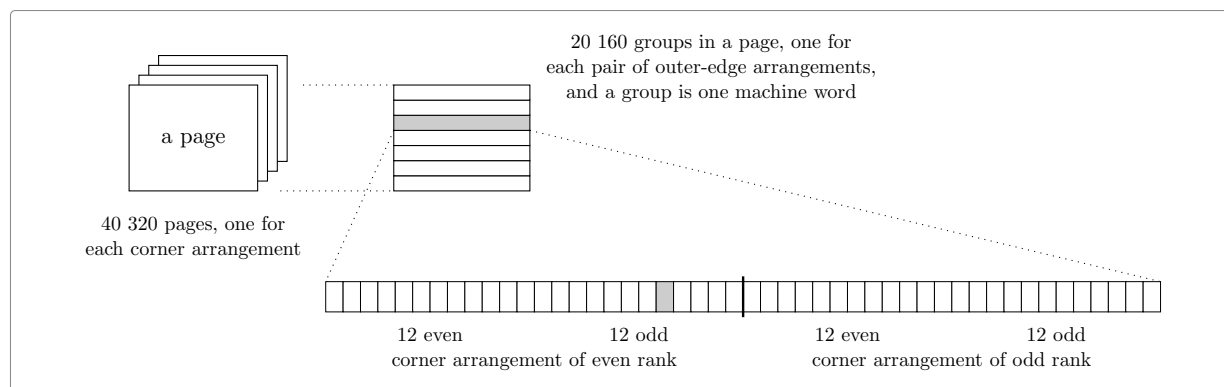


Figure 10: The map: a page per corner arrangement, a group for each pair of outer-edge arrangements and a bit for each arrangement of the 4 middle edges, the 12 even ones low and the 12 odd ones high.

5.4 Growing the map level by level

A map is sound at d when every bit set to 1 is a position within d moves of solved. The run starts from the empty map, with no bit set. It is sound at 0. Level d turns a map sound at $d - 1$ into a map sound at d . The bits already set stay set, and there are 2 ways to set new ones.

- The *search* lists every word of length d and sets the bit of each position of the coset it reaches. It is complete, but expensive.
- The *prepass* is a traversal of the map. For every position h that is set in the map, it directly sets the bit of hm for each of the 10 moves m of H . If xh is within $d - 1$ moves, then xhm is within d moves, and it is in the coset because h and m are in H . So the prepass keeps the map sound. It is

cheap, but not complete: it covers the words of length d whose last move is in H , and misses those that end with one of the other 8 moves.

These two operations work well together. If the prepass runs before the search, the search can restrict the last move of its words to the 8 moves that are not in H : the prepass has already set the bits of the others. Finally, the search becomes very expensive as d grows, typically beyond level 14. We follow Rokicki's strategy, which depends on how many bits the map holds:

- **While the map holds at most 6 million bits.** The search alone.
- **Once it holds more, up to level 16.** The prepass, then the search with the moves not in H for the last move of a word. Near the end of a word, it also only tries moves that strictly lower the distance. By near the end we mean that the number of moves left plus the distance is less than 5. Level 16 has a further refinement. If n bits are set after its prepass, the search stops once the map holds 167 million plus $\frac{n}{3}$ bits.
- **Levels 17 to 20.** The prepass alone.

These cuts can lose words: a word can waste a move early and still end in H . This is allowed. What is proved is that every bit set is correct, not that every word is found, so we do not require completeness.

In fact, the search does only a small part of the marking. Its 2 thresholds are small: the cuts start when the map is 0.03 % full, and the search at level 16 stops once the map is about 1 % full (167 million bits). The last 4 levels fill the rest with the prepass alone. A prepass sets up to 10 new bits for each bit already set, so it can multiply the number of bits set by up to 11, and 4 prepasses by up to 11^4 , more than 14 000. Most of the marking is done by the prepass.

This works because the prepass is cheap. Applying a move of H to a position of the coset does 3 separate things to the 3 numbers that represent it. The corner arrangement goes to another corner arrangement, so a page goes to a page. The outer-edge pair goes to another pair, so a group goes to a group. The middle arrangement goes to another middle arrangement, so the 24 bits of the group are rearranged among themselves. This rearrangement depends only on the move and the group. It is a table lookup and a shuffle of one machine word. So the prepass never takes a position apart. It never builds a cube, never ranks one, never looks a position up. For each of the 10 moves it reads the whole map and writes the whole map.

The prepass and the proof that it keeps the map sound are in `RowMap.v`. `RowLv1.v` has a faster version, where the array holding a page is fetched once and put back once instead of once a word, and proves it equal to the first.

5.5 The members left over

Stopping the search at level 16, and stopping level 16 early, leaves members the run never reaches. When the 2 runs ended, 32 bits of the 406 425 600 words were still clear. They are simply the members that no word the cut search kept, followed by moves of H , happened to reach. Searching deeper would leave fewer and cost hours; each one left over costs a line.

So we give each of the 32 a word of 20 moves by hand, in `RowWits.v`. Our prototype gave 28 of them, and the last 4 were found one at a time. None of that is believed. `RowWitsChk.v` applies each of the 32 words to its member and asks for the solved cube. If the producer had made a mistake, or given a word of 21 moves, or named the wrong member, the check would fail; it cannot make the theorem false. The replay is also the only thing in the development that tests the 2 ideas of which facelet is which – the prototype's, which found the words, and Rocq's, which builds the member – against each other.

The 32 are marked into the map at the very end, by `RowMark.v`, and not at the start. In a map sound at d , every bit set is within d . A witness is within 20, so seeding one before the run would make level one mark its neighbours as within one, and they are not. Marking at the end is sound because the map is already sound at 20 when the marks go in.

5.6 The run, and its one boolean

In `RowSrchn.v` the run is 10 lines. It carries the map, the map it reads while it writes and the number of bits set so far. This number decides whether the cuts are on.

```
Fixpoint runskn (n : nat) (d : nat) (n0 : int) (m dst : rmap) : rmap :=
  if n is n1.+1 then
    if Uint63.ltb ncutb n0 then
      let m1 := prep m dst in
      let mn := slvlskn true d.+1 m1 (mcount m1) in
      runskn n1 d.+1 mn.2 mn.1 m
    else
      let mn := slvlskn false d.+1 m n0 in
      runskn n1 d.+1 mn.2 mn.1 dst
  else m.
```

With the cuts on, a level is the prepass `prep`, then the search `slvlskn`. With the cuts off, it is the search alone. The search adds one to the count for each bit it sets. Counting the bits of the whole map, `mcount`, is done only after a prepass, which reads the whole map anyway.

Everything above is one boolean. `RowFoldCubDef.v` builds the map, runs the 20 levels, marks the 32 and asks whether every bit is set:

Definition `rowfulli0 : bool := mfullf ffuli ycwitsoi0.`

Lemma `rowfulli0T : rowfulli0 = true.`

Proof. `Time native_cast_no_check (erefl true).` `Time Qed.`

That `Qed` is the 50 minutes. The file that holds it, `RowFoldCubBool.v`, is one **Require** and one **Lemma**: a file that runs loads no proof, and a file that proves runs nothing. Everything else – that the prepass keeps the map sound, that the search marks only members, that a full map is the theorem – is proved beside it, of a map and a run the proof never evaluates.

5.7 Folding the map

The map can be made smaller, and we made it smaller to see what its size does to the run. Both maps were searched, the folded one and the unfolded one, and the 2 runs are compared at the end of the section.

The map is 40 320 pages, one for each arrangement of the corners.

The fold works because of the position we chose. Of the 48 symmetries of the cube, 16 keep the top and bottom faces in place. Each of them sends the 10 moves to the 10 moves, so it maps H to itself, and each leaves the superflip unchanged, so it maps the superflip's coset to itself. A member and its image under such a symmetry need the same number of moves, so 2 pages related by a symmetry hold the same answer. One page of each family is then enough: 2 768 of the 40 320, a factor of 14.6. A level of the run is one pass over the map, so there is 14.6 times less of it to walk. The price is undoing a symmetry whenever a kept page is read.

The kept pages go in pairs of their own, and a pair shares one word as it does on the unfolded side: 1 496 words in place of 2 768. There are 224 pages that are their own partner and use half a word.

The fold has to be proved as well as written. A symmetry sends a member of the coset to a member of the coset. Undoing it gives back the position the page stood for. A map sound after one level is sound after the next. That is the largest single part of the coset's proof.

5.8 The reused and the new

We reuse nearly everything. The abstract search and its contract come over unchanged, and so do the cube, the permutations, the machine-integer tools, the phase one table and its generator. Each of the new tables has a check of its own, in a file of its own, so that a failure names the table that failed.

There are 5 new things, and only one of them is a search.

- The rank and the sign of a permutation. Rocq's library has both, but for permutations that do not compute, so we need their effective version on machine integers.
- The link between a position and its 3 ranks. The map's bits are triples of numbers, so what the run proves is a statement about triples. To read it as a statement about the cube we have to turn a triple back into the position it stands for. The other way we already had.
- The pass that steps a whole map one move at a time.
- The fold by the 16 symmetries.
- Words given by hand for the members the run does not reach.

5.9 The files and the cost

The coset adds 86 hand-written files and 23 800 lines, besides the generated tables. The coset and its members are `Row.v`, `RowMemb.v` and `RowInst.v` the ranking and the moves `Lehmer.v` with the `RowPart`, `RowMove` and `RowTab` groups; the map and the search `RowMap.v`, `RowRun.v`, `RowSrch.v` and `RowFinal.v` the fold the `RowFold` group; and the 2 runs the `RowCub` and `RowFoldCub` groups, ending in `RowCubDone.v` and `RowFoldCubDone.v`. `README.md` lists every one of them with what it does.

The search ran twice, over the folded map and over the unfolded one, with the same search in both, and both times it filled the map.

the run	wall clock	processor time
over the folded map	50 min	50 min
over the unfolded map	2 h 32	2 h 31

The fold is worth 3 times on the wall clock and 3 times on processor time. The map is 13 times smaller, so the run does not follow the size of the map. The map sets the memory. The unfolded map is 3.25 GB against 248 MB, and a level reads one map while it writes the other.

The phase one table is 2.9 GB of Rocq source and 4.5 GB once checked, 8 h 48 of processor time. It is generated once and shared with the lower-bound work.

The statement has no hypothesis left.

Theorem `real_superflip_row_fold_run0 h :`
`h \in H -> superflip^-1 * h \in ball Sset 20.`

H is the subgroup above, and the superflip is its own inverse, so every position of the superflip's coset is within 20 moves.

6 Conclusion

We prove 2 lower bounds. No position of the cube is solved in 19 face turns, and none in 25 quarter turns. In each case one position is the witness, the superflip for the first and the four-spot with the superflip on it for the second. In each case the proof is a search that comes back empty over a table of estimated distances.

We do not prove that 20 and 26 always suffice. The upper half is a different computation. It is not one search that finds nothing, but 2 billion searches that must each find everything. We did one of the 2 billion, and the section above gives its cost.

The three share a trunk. The cube itself, as permutations of the 48 facelets, with the 18 moves and the group they generate. Balls, the positions within d moves. The abstract search, whose contract is that a false answer is a proof. The rule that any summary of a position, together with any table that passes one check, gives an estimate that is never too big. And the tables themselves, held as machine integers in arrays rather than as lists of unary numbers.

What each of the three needed of its own:

- **The 20 face turns.** The phase one summary, which is the edge flips and the slice, its table and the certificate that checks the table. The same search seen from 3 angles, and 30 pieces run side by side.
- **The 26 quarter turns.** The cut to 6 prefixes, which is the one piece of the development argued by hand rather than computed, and the parity argument that turns 25 into 24. A second and much larger summary, 29 billion values, with a table of its own and a check of its own.
- **One coset.** A coset held as a map of bits rather than as a tree of positions. That needs 5 new pieces. The rank and the sign of a permutation on machine integers. The link between a position and its 3 ranks. A pass that steps a whole map one move at a time. The fold by the 16 symmetries. And words given by hand for the members the run does not reach.

The same search written in OCaml is about 4.5 times faster than the one Rocq runs. We ran both at radius 19 on the reference machine. The OCaml program visits 137 607 893 106 positions in 19.2 processor-hours, which is 0.50 microseconds a position. Rocq takes 89.5 processor-hours over the same tree, which is 2.34. A factor of **4.7**.

We do not assume that the two walk the same tree. We divide each of the 30 Rocq pieces by the positions its OCaml counterpart visited. The result is between 1.85 and 2.86 microseconds, over pieces that differ in size by a factor of 2.8. So the run takes a night because the tree holds 138 billion nodes, not because the prover is slow: in OCaml the same tree still costs 19 processor-hours.

The whole development, counted in hand-written Rocq and leaving out the generated tables, is 36 300 lines. Each line of the table counts what that piece adds to the ones above it.

	files	lines
the superflip, for the 20 face turns	44	12 600
the four-spot, for the 26 quarter turns	19	6 100
one coset of the upper bound	70	17 600
in all	133	36 300

This development was written with the help of Claude, Anthropic’s coding assistant.

The sources are at github.com/thery/DoubleCover/code/Rubik, with the note, its figures and Reid’s transcribed post beside them.

References

1. The Rocq Development Team: The Rocq Prover, <https://rocq-prover.org/>
2. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo, 2021, <https://doi.org/10.5281/zenodo.4457887>
3. Korf, R.E.: Depth-first Iterative-deepening: An Optimal Admissible Tree Search. *Artificial Intelligence*. 27, 97–109 (1985). [https://doi.org/10.1016/0004-3702\(85\)90084-0](https://doi.org/10.1016/0004-3702(85)90084-0)
4. Culberson, J.C., Schaeffer, J.: Pattern Databases. *Computational Intelligence*. 14, 318–334 (1998). <https://doi.org/10.1111/0824-7935.00065>
5. Korf, R.E.: Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In: *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*. pp. 700–705 (1997)
6. Kociemba, H.: The Two-Phase Algorithm, <https://kociemba.org/cube.htm>
7. Reid, M.: superflip composed with four spot. Post to the Cube-Lovers mailing list, 2 August 1998, The address cited by Rokicki et al. no longer resolves; this is a transcription, <https://github.com/they/DoubleCover/blob/main/doc/reid-1998-fourspt.md>